

CHALLENGE #11: Byp4ss3d Write-up.

byp4ss3d

Medium

Web Exploitation

picoMini by CMU-Africa

AUTHOR: YAHAYA MEDDY

Description

A university's online registration portal asks students to upload their ID cards for verification. The developer put some filters in place to ensure only image files are uploaded but are they enough? Take a look at how the upload is implemented. Maybe there's a way to slip past the checks and interact with the server in ways you shouldn't.

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints ?

1

2

Apache can be tricked into executing non-PHP files as PHP with a `.htaccess` file.

3,833 users solved

picoCTF{FLAG}

Hints ?

1

2

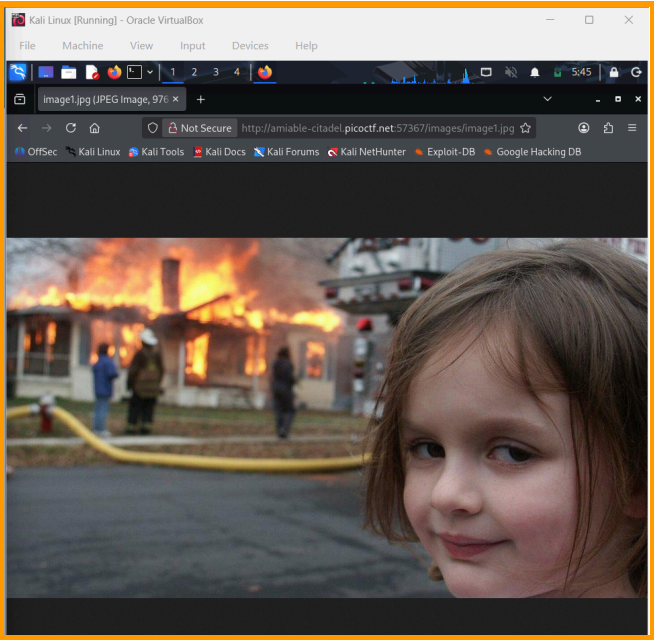
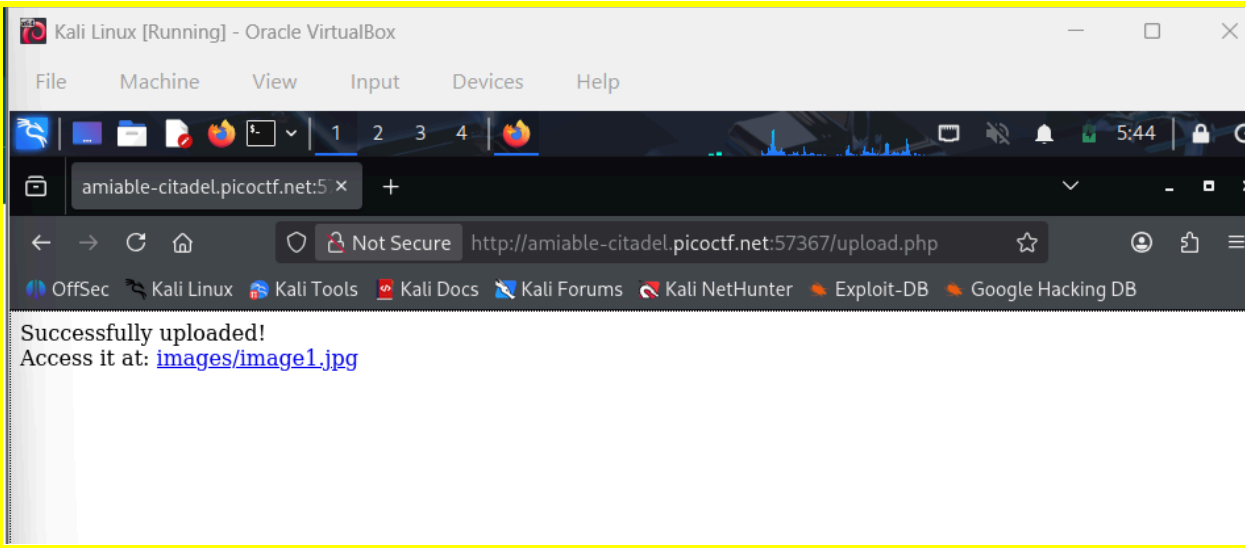
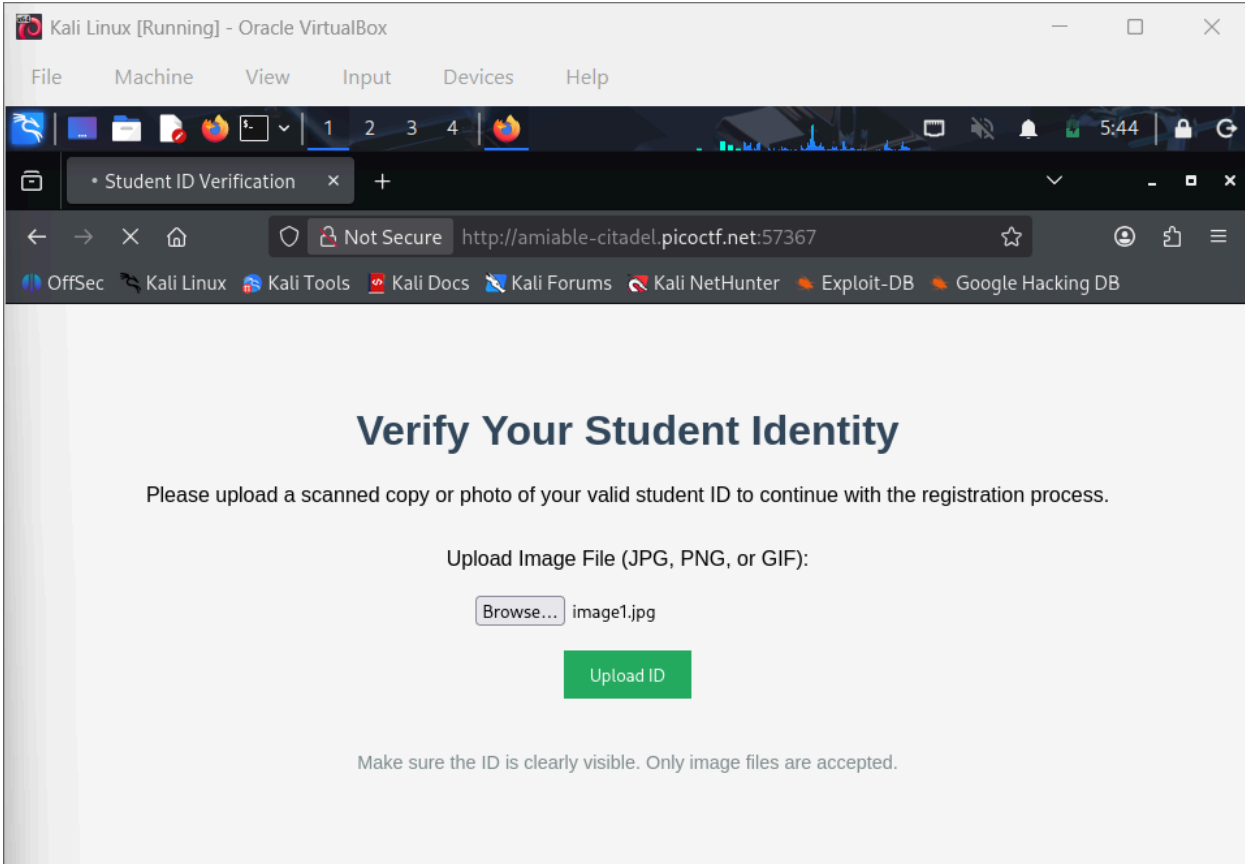
Try uploading more than just one file.

Flag

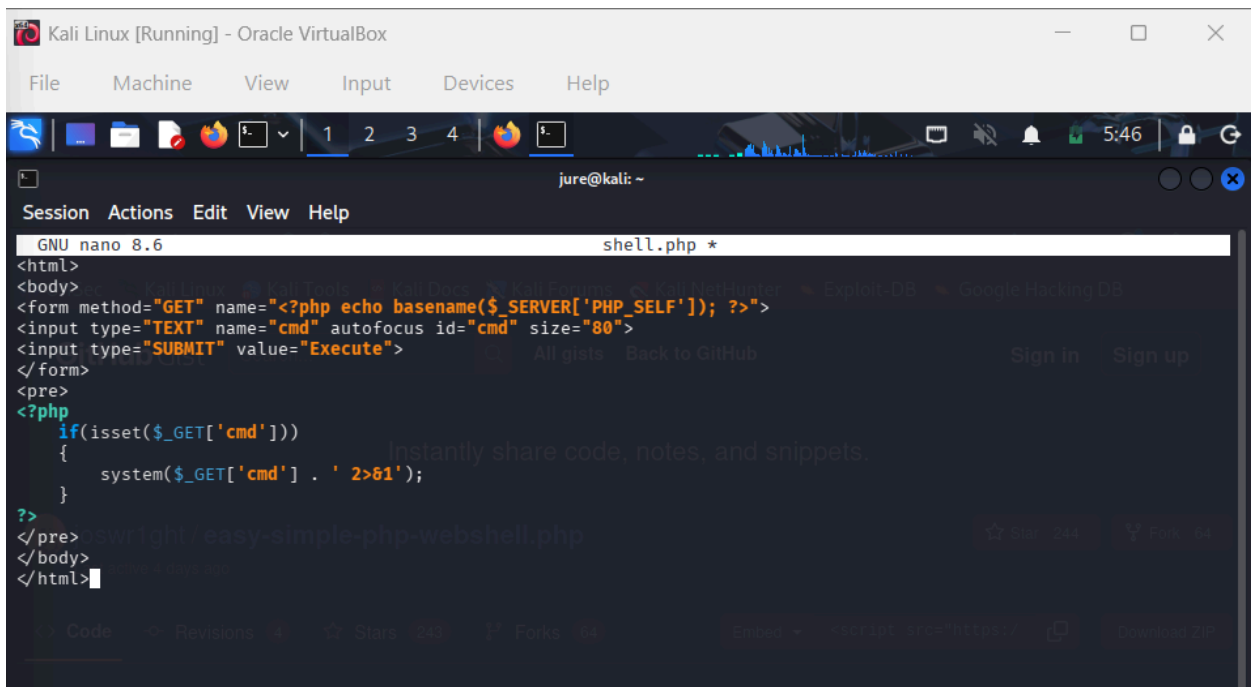
Solving Process:

The website displayed a page where users could **verify their student identity by uploading an image file**, with the system only allowing image formats such as JPG, PNG, or GIF. The goal of the challenge was to bypass this restriction and ultimately retrieve the flag.

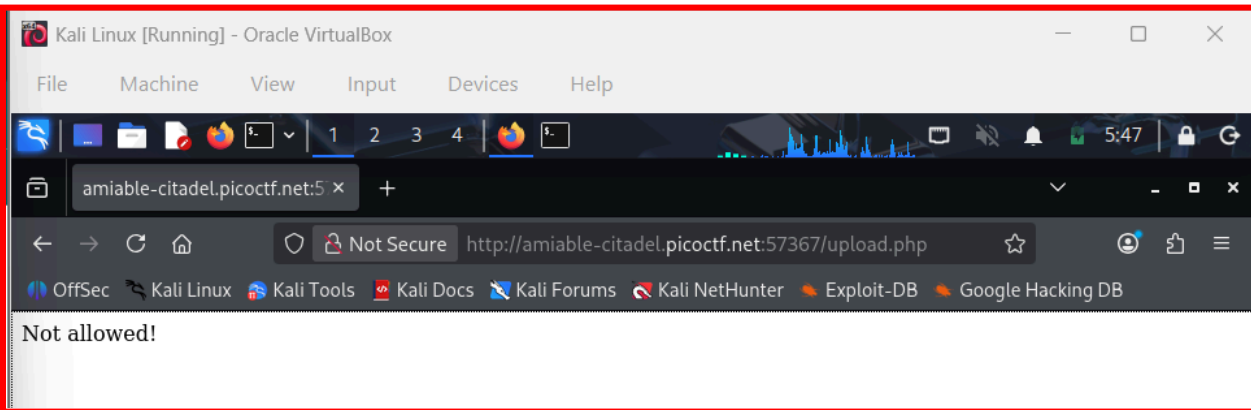
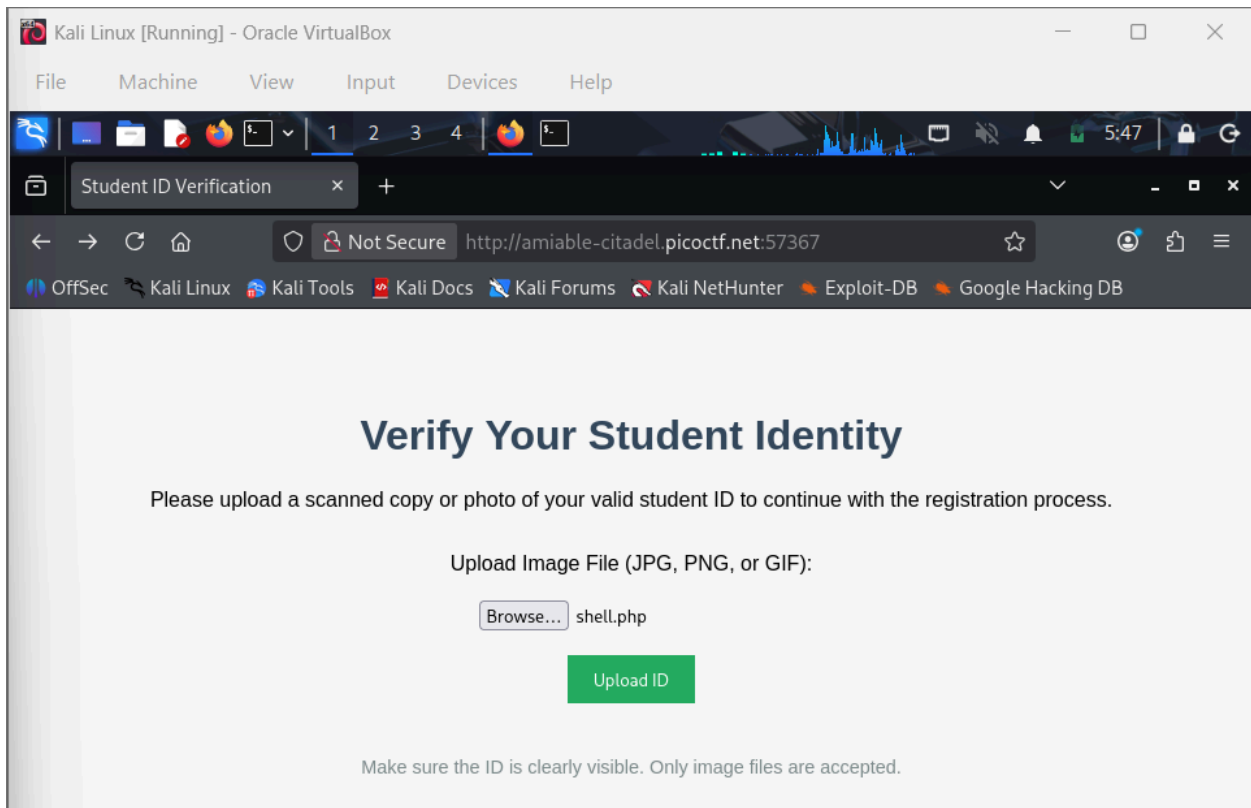
To understand how the upload system worked, a normal PNG image was first selected using the **Browse** button and uploaded to the site. After clicking Upload ID, the application confirmed that the file was successfully uploaded and provided a link where the uploaded image could be viewed. Opening the link confirmed that the image was accessible on the server.



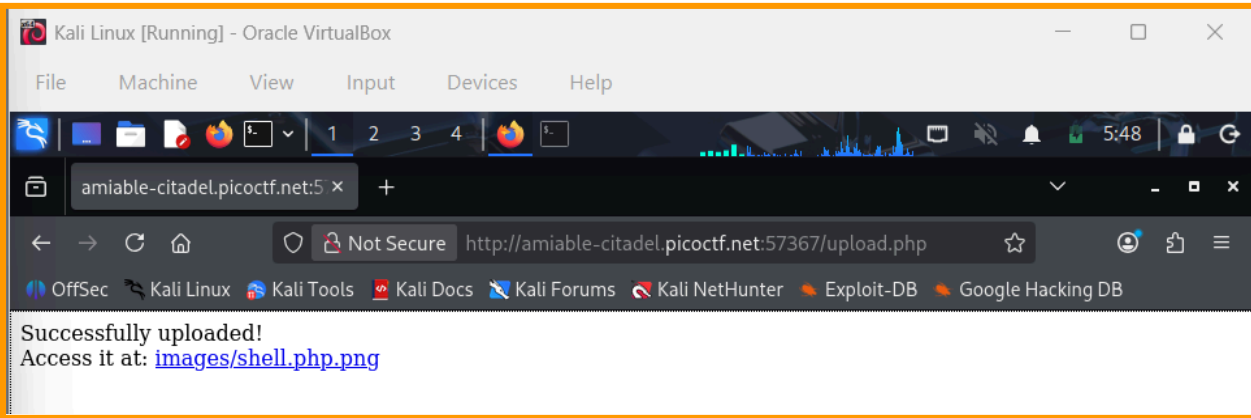
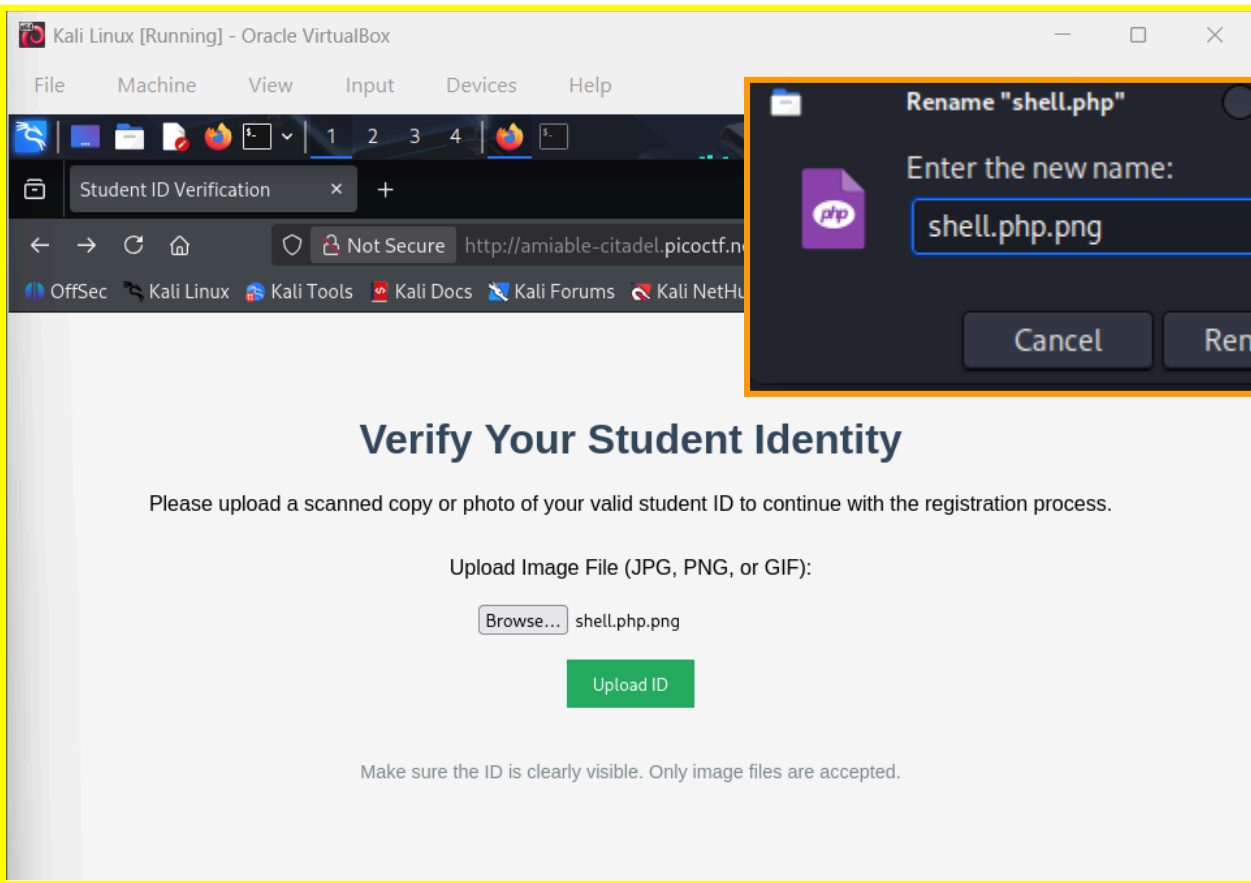
After confirming the upload process, the next step was to attempt uploading a malicious file. A simple **PHP web shell** was obtained and saved locally as **shell.php**. This file contained code that would allow commands to be executed on the server.



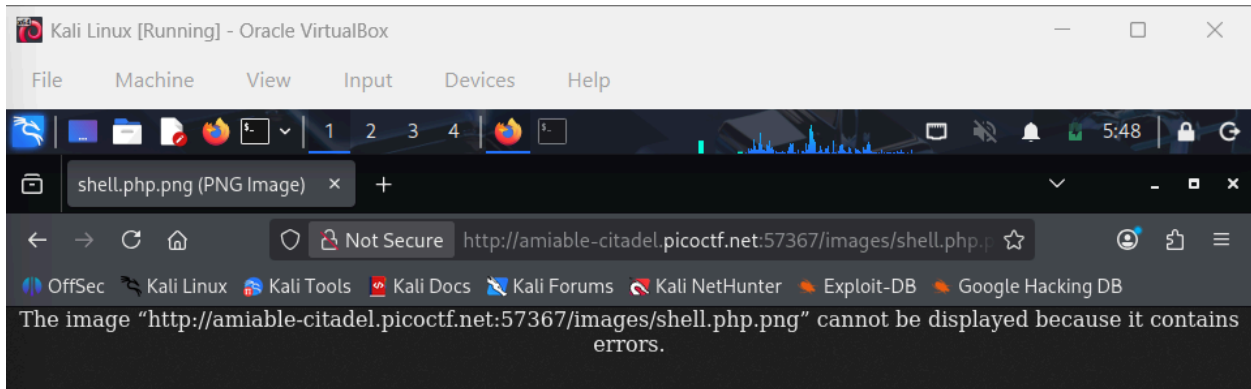
However, when attempting to upload the file directly, the server rejected it because **PHP files were not allowed**.



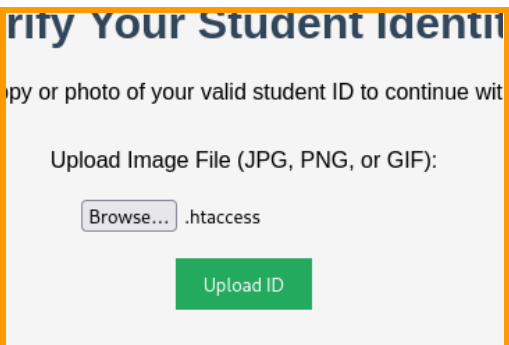
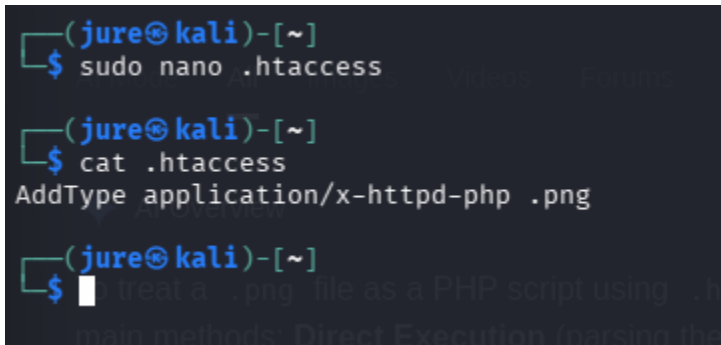
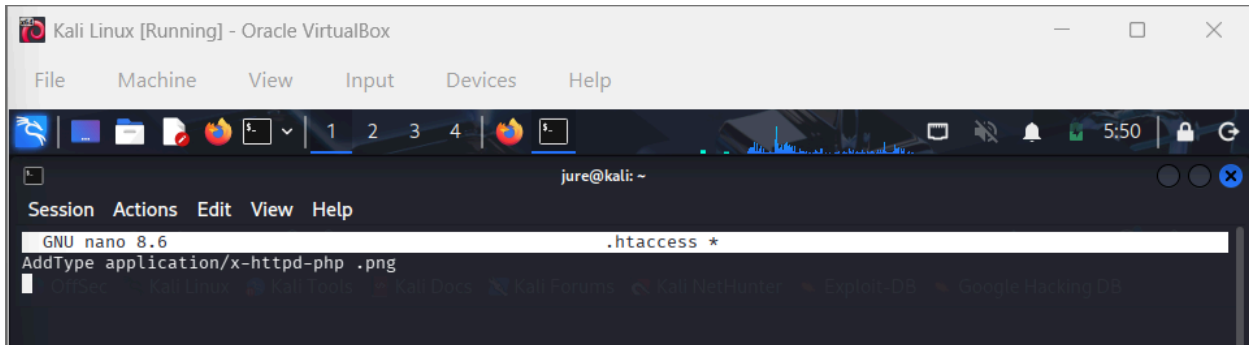
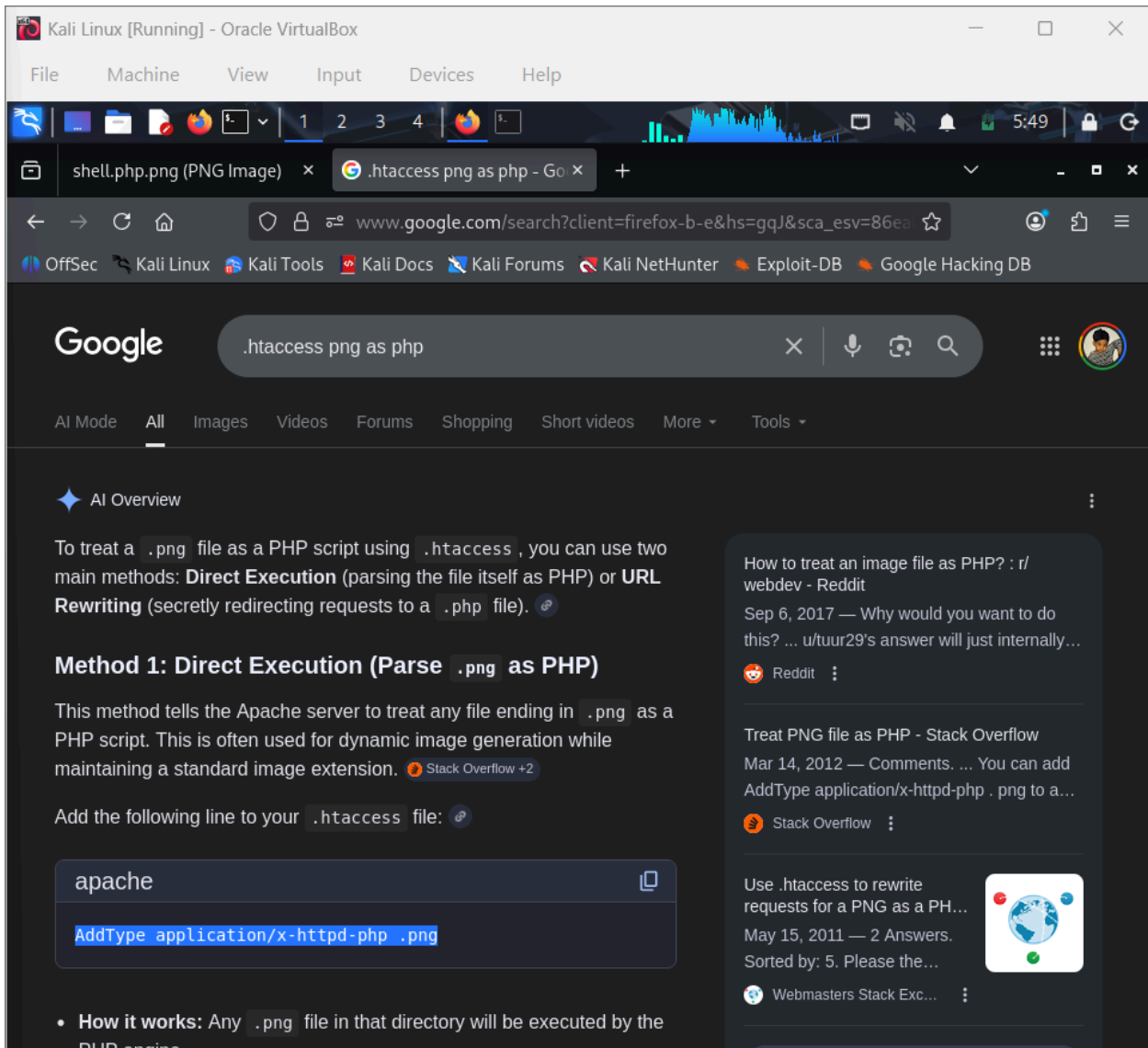
To bypass the upload restriction, the file was renamed to **shell.php.png**. This technique works because some file upload filters only check the file extension, and by adding **.png** to the filename, the server may incorrectly treat the file as an image. After renaming the file, it was uploaded again, and this time the server accepted the file successfully.



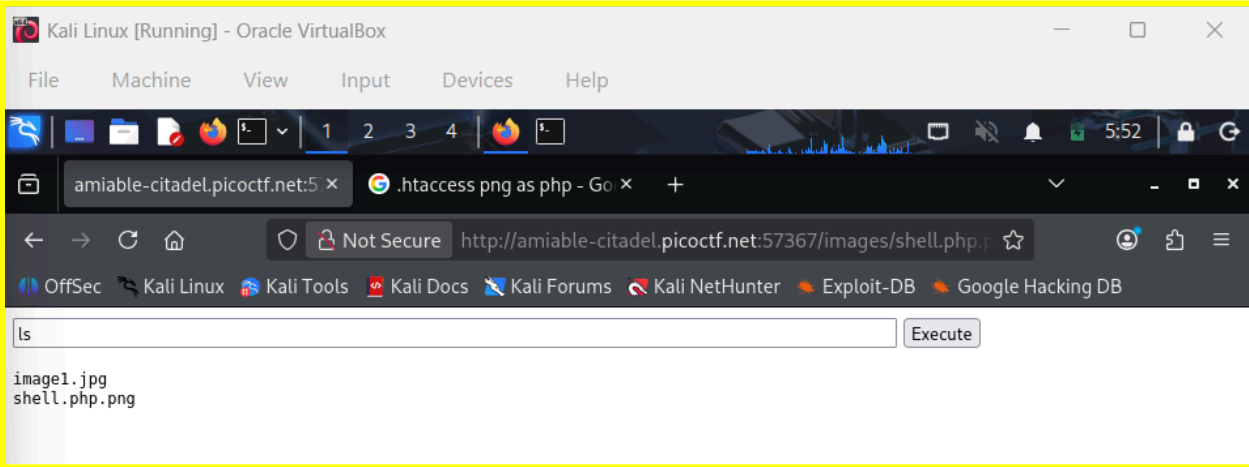
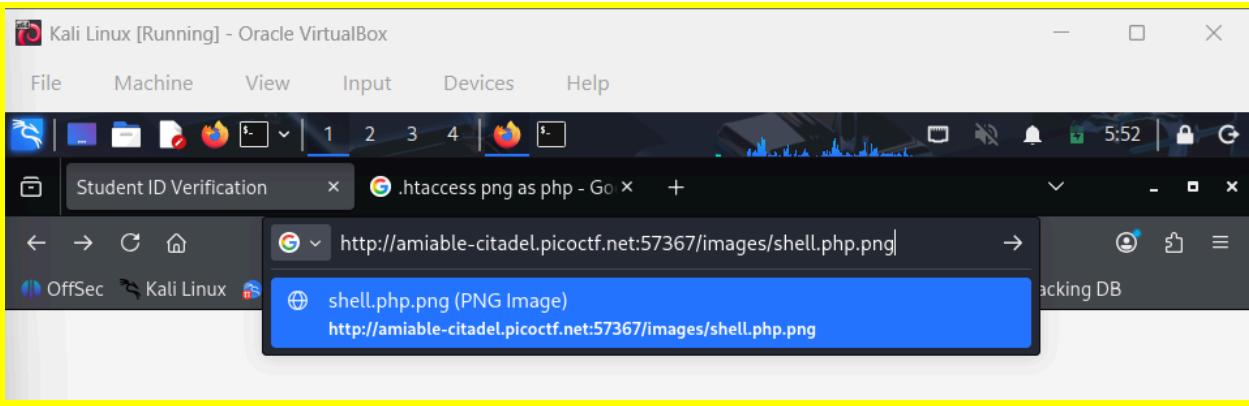
When attempting to open the uploaded file, the browser displayed an error stating that the image could not be displayed because it contained errors. At this point, the challenge hint suggested that **Apache can be tricked into executing non-PHP files as PHP using an .htaccess file**.



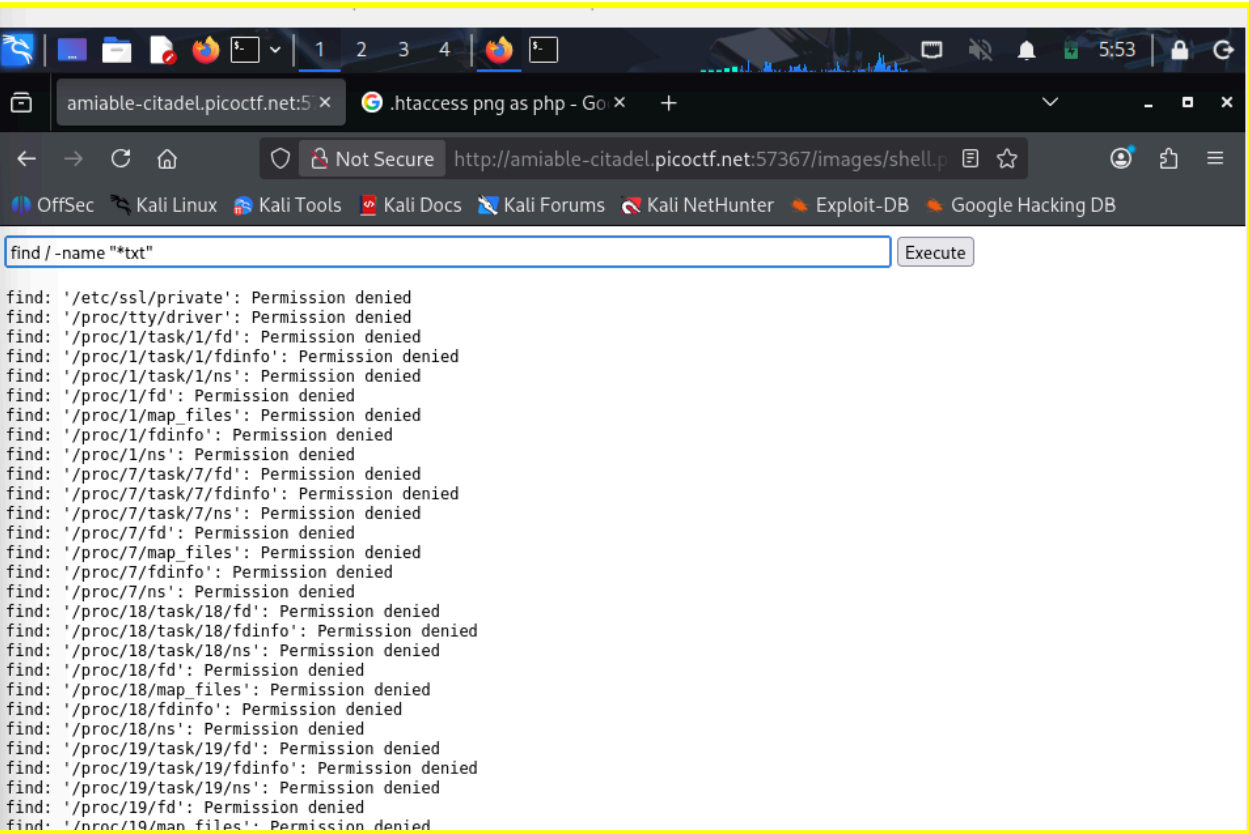
To take advantage of this, a new file named **.htaccess** was created. This configuration file contained instructions that would force the server to treat PNG files as PHP scripts. After creating the **.htaccess** file, it was uploaded to the same directory as the malicious file.



Once the configuration file was uploaded, the previously uploaded **shell.php.png** file was accessed again through the images directory in the browser. This time, the web shell interface appeared, allowing commands to be executed directly on the server.

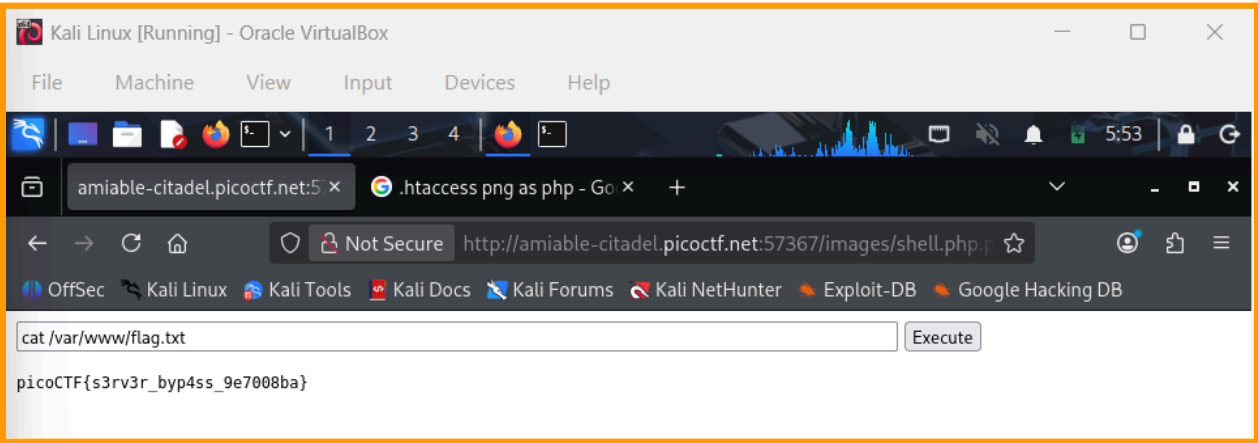


Using the web shell, commands were executed to explore the system. The **ls** command was first used to list the files in the current directory. To locate the flag, the **find** command was executed to search the system for files ending with **.txt**. Among the results, a file named **flag.txt** was identified along with its full path.



```
/usr/share/perl/5.32.1/Unicode/Collate/allkeys.txt
/usr/share/perl/5.32.1/Unicode/Collate/keys.txt
/usr/share/perl/5.32.1/Unicode/Blocks.txt
/usr/share/perl/5.32.1/Unicode/NamedSequences.txt
/usr/share/perl/5.32.1/Unicode/SpecialCasing.txt
find: '/var/cache/apt/archives/partial': Permission denied
find: '/var/cache/ldconfig': Permission denied
/var/www/flag.txt
find: '/challenge': Permission denied
```

Finally, the **cat command** was used with the file's full path to display its contents. The flag appeared on the screen and was then copied and submitted to successfully complete the challenge.



byp4ss3d

Medium Web Exploitation picoMini by CMU-Africa

AUTHOR: YAHAYA MEDDY

Description

A university's online registration portal asks students to upload their ID cards for verification. The developer put some filters in place to ensure only image files are uploaded but are they enough? Take a look at how the upload is implemented. Maybe there's a way to slip past the checks and interact with the server in ways you shouldn't.

You can access the web application at [here!](#)

This challenge launches an instance on demand.

Its current status is: **RUNNING**

Instance Time Remaining: **14:04**

Restart Instance

Hints

1 2

3,833 users solved

96% Liked

picoCTF{s3rv3r_byp4ss_9e7008ba}

Submit Flag

Hurray! You solved this challenge.

Reflection:

This challenge demonstrated how weak file upload restrictions can be bypassed to achieve remote code execution on a server. By manipulating file extensions and using an .htaccess configuration file, it was possible to trick the server into executing a malicious script disguised as an image. The activity highlighted the importance of strict server-side validation, secure file handling, and proper configuration management to prevent attackers from exploiting file upload features.